Extração e Preparação de Dados

Aula 12 – Escalonamento e Desbalanceamento

ibmec

Quem sou eu?

Professor: Luís Aramis dos Reis Pinheiro.

- **Doutorado e Mestrado** em **Ciências Mecânicas** – **UnB** – **CAPES 7**
- **Graduação** em Licenciatura em Física – **UNIFAP**

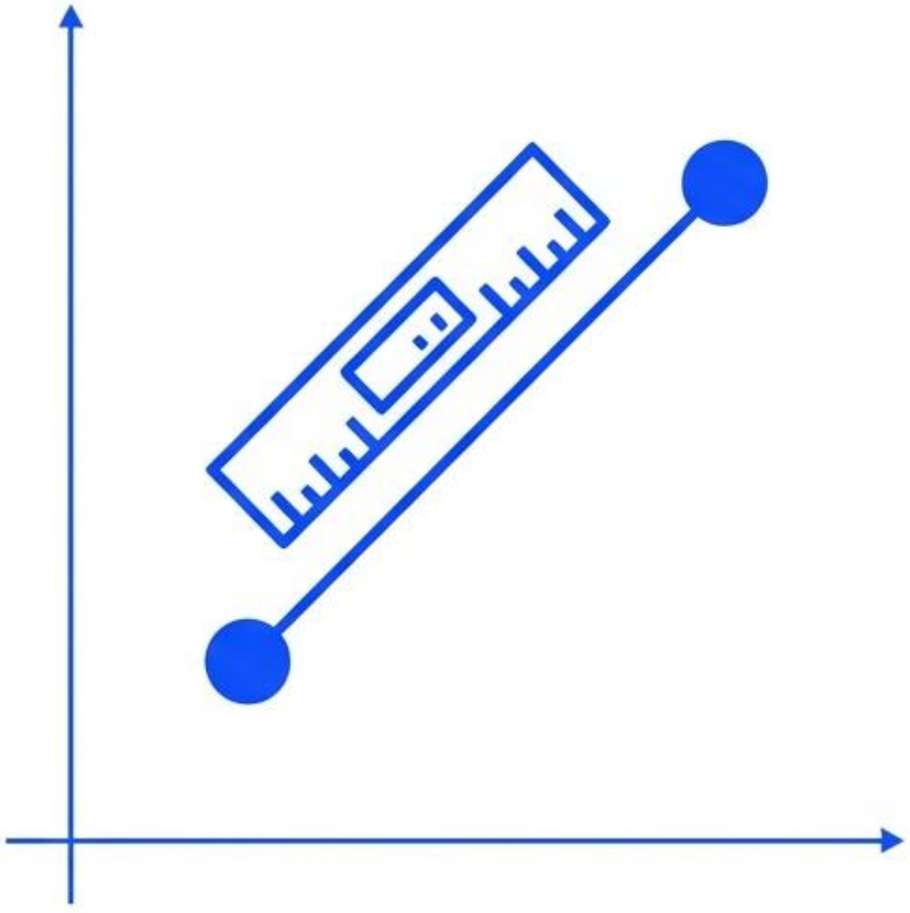
luis.pinheiro@professores.ibemec.edu.br

(96) 99907-5819

@l_aramis

Escalonamento e Desbalanceamento

A Matemática das Distâncias e Classes Minoritárias.



- Disciplina: Extração e Preparação de Dados (IBM8915)
- Data: 24/03 (Terça-feira)

Contexto: Os dados estão estruturados e sem nulos. Hoje, faremos os ajustes finais exigidos pelo Scikit-Learn para garantir que a máquina não confunda grandeza com importância.

O Viés da Grandeza Matemática

A Ilusão da Máquina

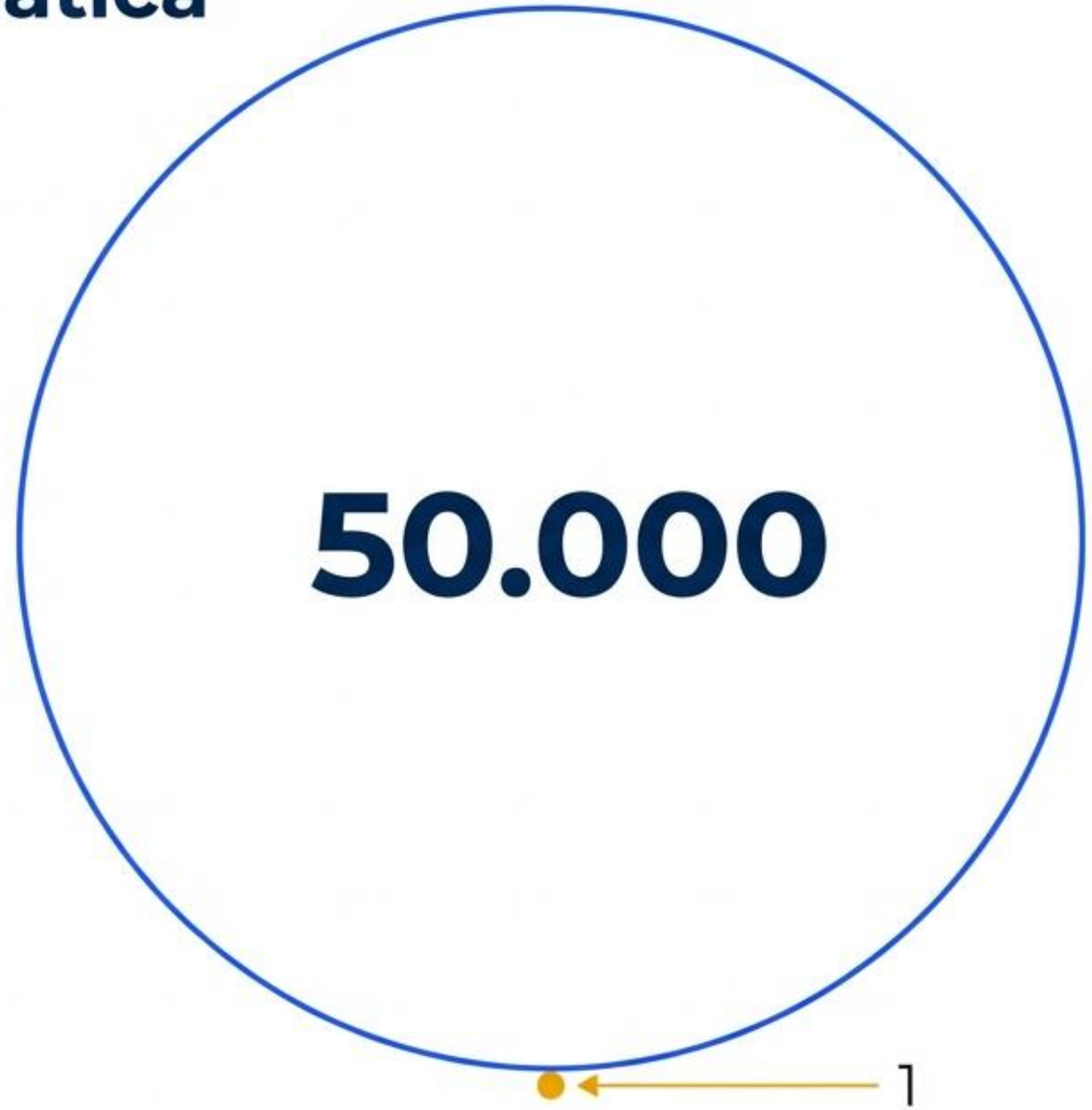
Algoritmos baseados em distância (como KNN) interpretam números absolutos de forma literal.

Choque de Dimensões

A comparação direta entre a Variável Salário (R$ 50.000) e a Variável Filhos (1) cria uma assimetria fatal.

A Consequência

O algoritmo assumirá que a renda é 50 mil vezes mais importante para a decisão matemática do que a composição familiar.



Diagnóstico: Precisamos tirar o ‘peso’ da moeda e deixar apenas o padrão matemático.

Nivelando o Campo: Feature Scaling

O Objetivo

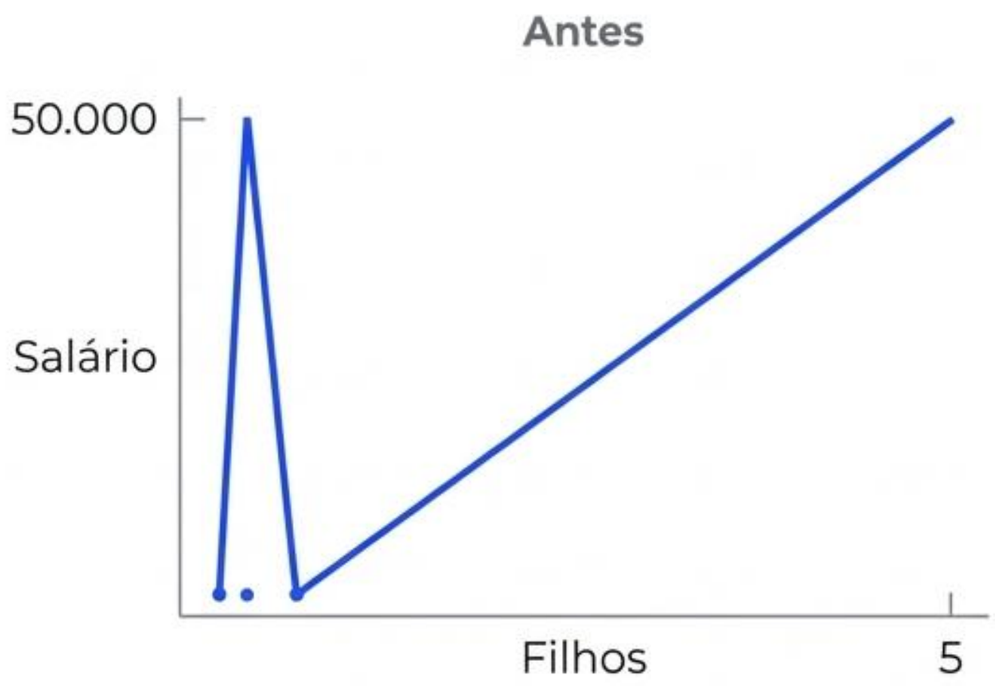
Comprimir todas as variáveis para a mesma escala sem perder a distribuição estatística.

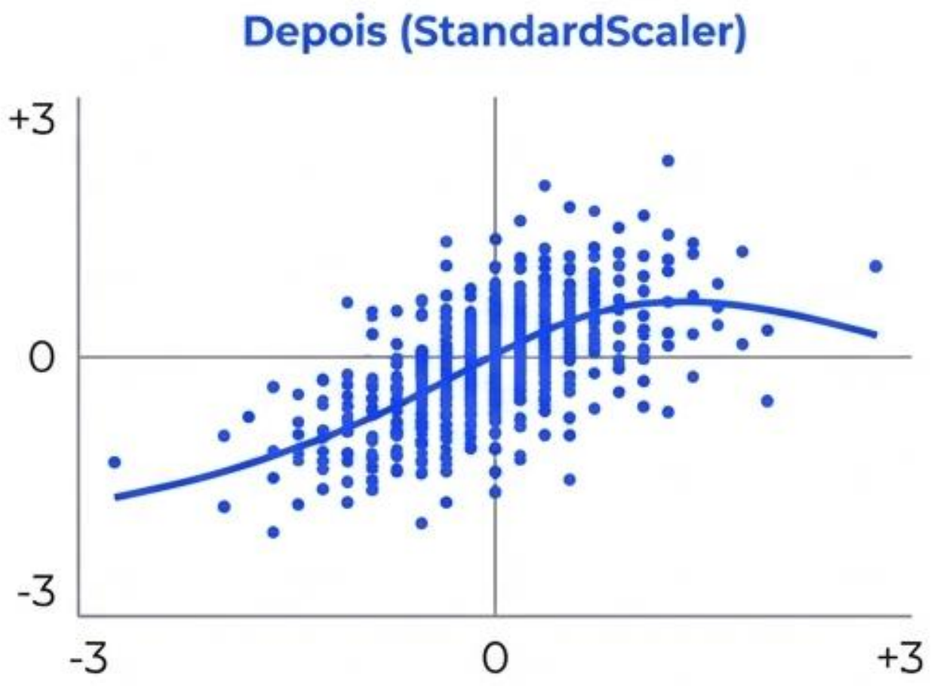
Padronização (StandardScaler)

Centraliza os dados na média 0 e desvio padrão 1 (ideal para dados normais).

Normalização (MinmaxScaler)

Espreme rigorosamente os dados num limite fixo entre 0 e 1.





Insight: Salário e número de filhos passam a falar a mesma língua matemática. Ninguém mais esmaga ninguém.

guem.

Nivelando o Campo: Feature Scaling

```python
from sklearn.preprocessing import StandardScaler,
MinMaxScaler

# Padronização (Média 0, Desvio Padrão 1)
scaler_std = StandardScaler()

# Normalização (Limites rígidos entre 0 e 1)
scaler_minmax = MinMaxScaler()
```

— O Objetivo

Comprimir todas as variáveis para a mesma escala sem perder a distribuição estatística.

— Padronização (StandardScaler)

Centraliza os dados na média 0 e desvio padrão 1 (ideal para dados normais).

— Normalização (MinmaxScaler)

Espreme rigorosamente os dados num limite fixo entre 0 e 1.

Prevenção de Data Leakage

O Erro Fatal

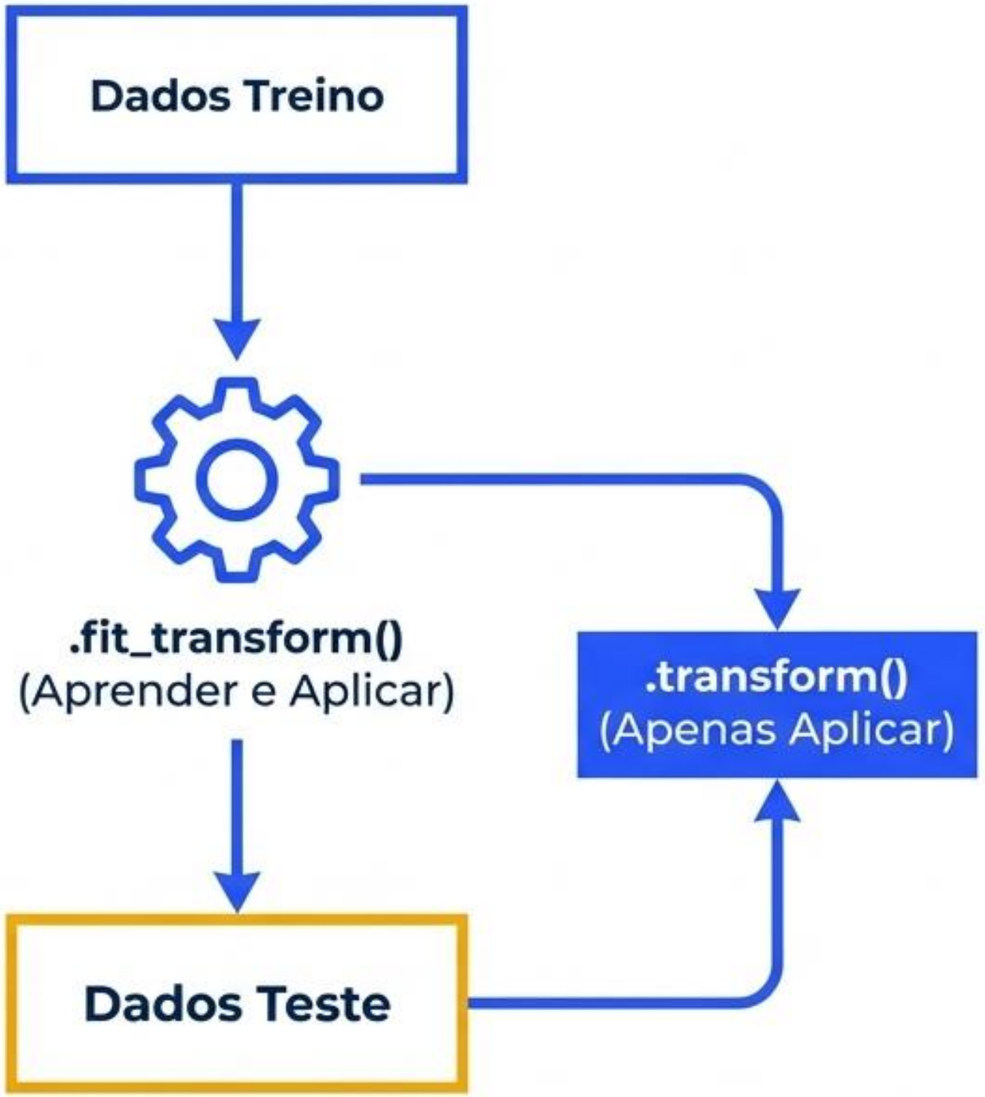
Escalonar o dataset inteiro antes de separar em Treino e Teste. Isso permite que o modelo calcule médias baseadas em dados do futuro.

Regra do Treino

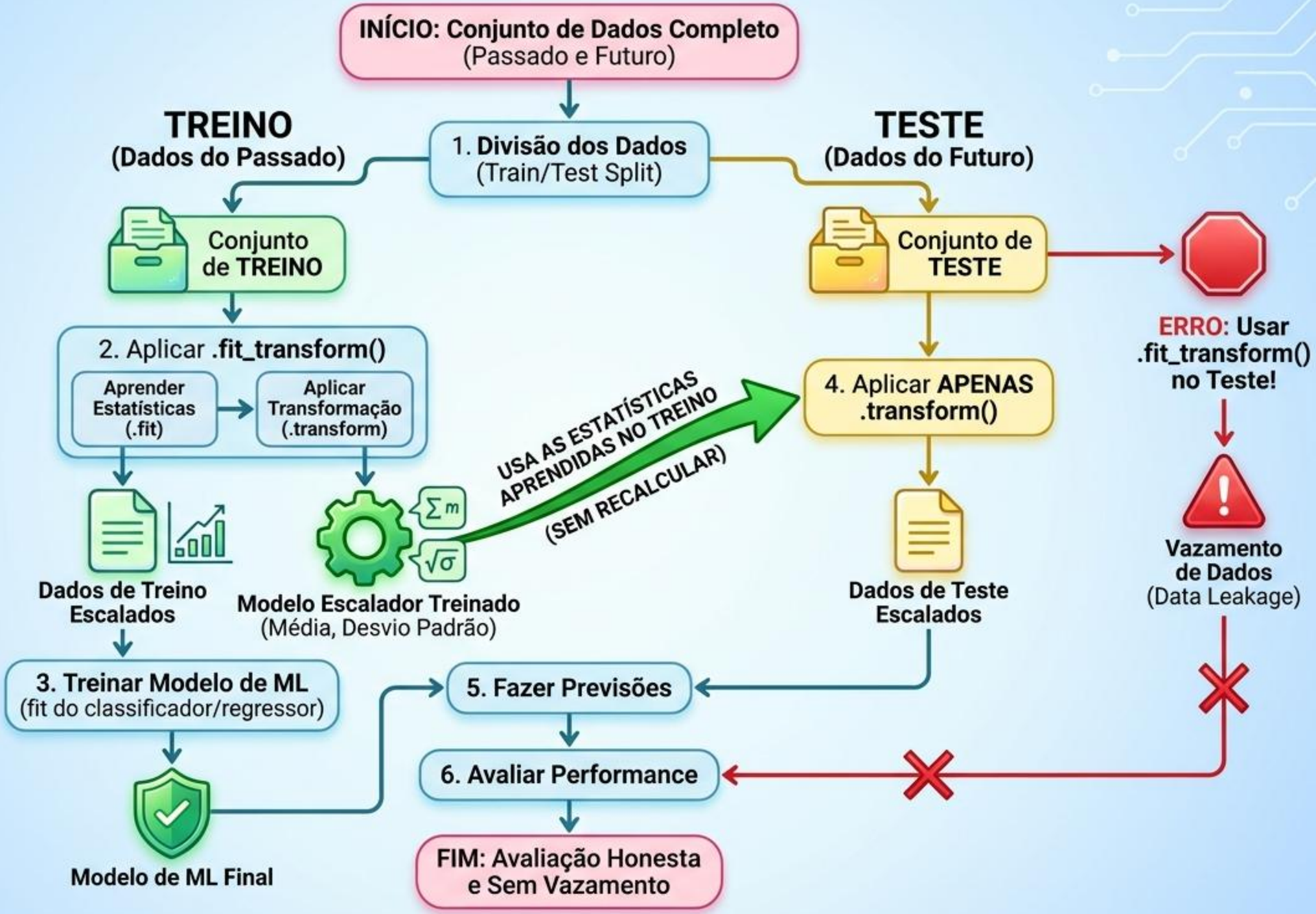
Usar .fit_transform() para aprender a régua (média/desvio) e aplicá-la.

Regra do Teste

Usar apenas .transform(). O teste deve ser medido rigorosamente pela régua do passado.



Regra de Ouro: O .fit() só enxerga o treino!


INÍCIO: Conjunto de Dados Completo
(Passado e Futuro)
TREINO
(Dados do Passado)
1. Divisão dos Dados
(Train/Test Split)
TESTE
(Dados do Futuro)
Conjunto de TREINO
Conjunto de TESTE
2. Aplicar .fit_transform()
Aprender Estatísticas (.fit)
Aplicar Transformação (.transform)
USA AS ESTATÍSTICAS APRENDIDAS NO TREINO
(SEM RECALCULAR)
4. Aplicar APENAS .transform()
ERRO: Usar .fit_transform() no Teste!
Dados de Treino Escalados
Modelo Escalador Treinado
(Média, Desvio Padrão)
Dados de Teste Escalados
Vazamento de Dados
(Data Leakage)
3. Treinar Modelo de ML
(fit do classificador/regressor)
5. Fazer Previsões
6. Avaliar Performance
Modelo de ML Final
FIM: Avaliação Honesta e Sem Vazamento

O Paradigma do Desbalanceamento

A Armadilha da Maioria

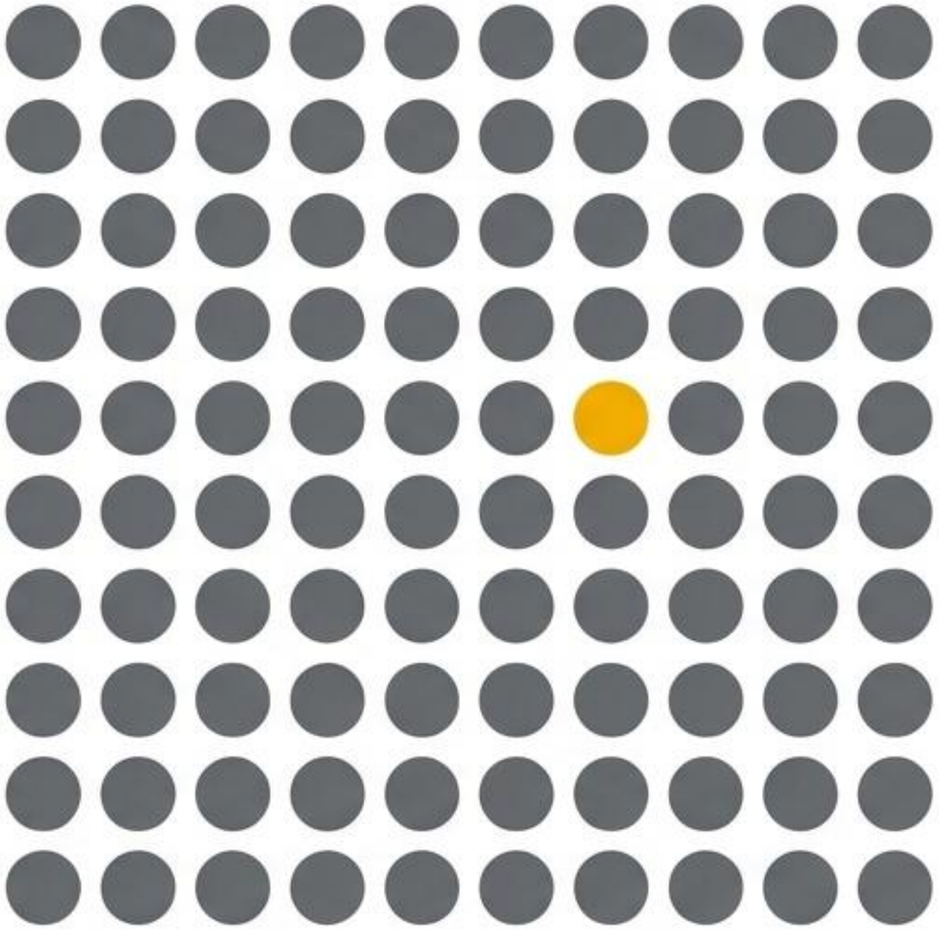
Identificar anomalias (1% de fraudes) em uma base de milhões de transações normais.

A Ilusão da Acurácia

Um modelo que simplesmente chuta “Nenhuma Fraude” acertará 99% das vezes, mas falhará categoricamente em seu único propósito.

A Solução

Não preencher de forma ingênua, mas reequilibrar as forças numéricas fornecendo exemplos suficientes do perfil fraudador.



Representação visual: 99% transações normais vs. 1% fraude isolada.

Dados Sintéticos: SMOTE

Biblioteca Alvo

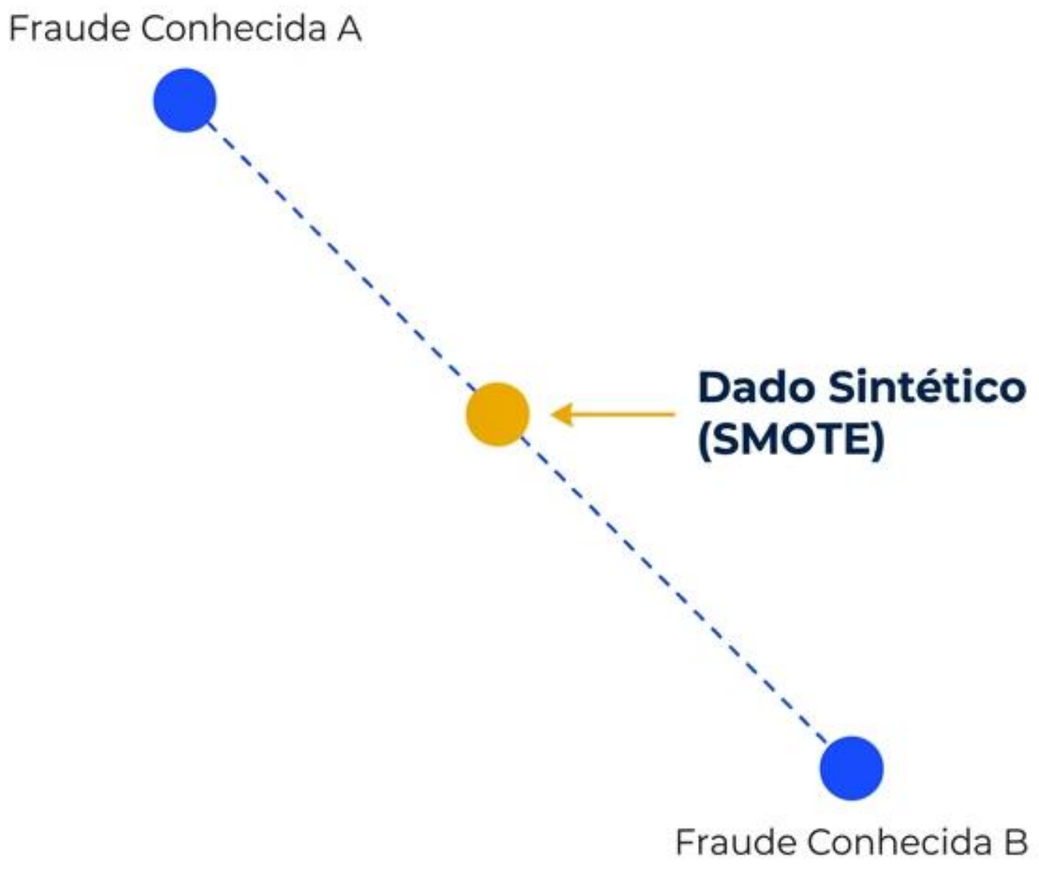
Utilização da biblioteca imbalanced-learn no ecossistema Python.

O que é o SMOTE

Synthetic Minority Over-sampling Technique. Criação matemática de novos dados, evitando o simples copiar e colar.

A Mecânica

Em vez de apenas clonar linhas (o que gera overfitting), o algoritmo cria novos dados sintéticos interpolando vizinhos da classe minoritária.



Dados Sintéticos: SMOTE

Biblioteca Alvo

Utilização da biblioteca imbalanced-learn no ecossistema Python.

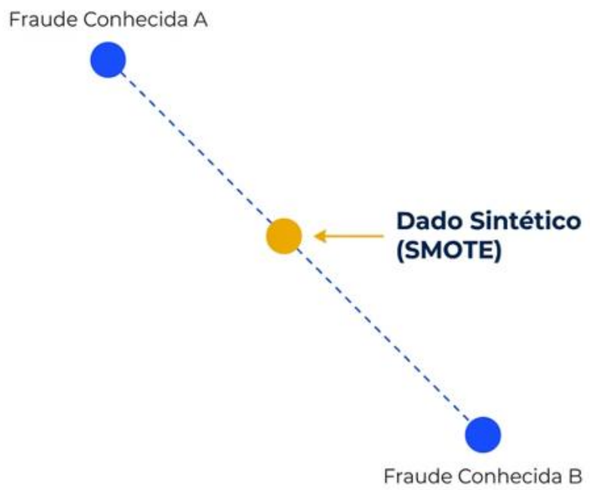
O que é o SMOTE

Synthetic Minority Over-sampling Technique. Criação matemática de novos dados, evitando o simples copiar e colar.

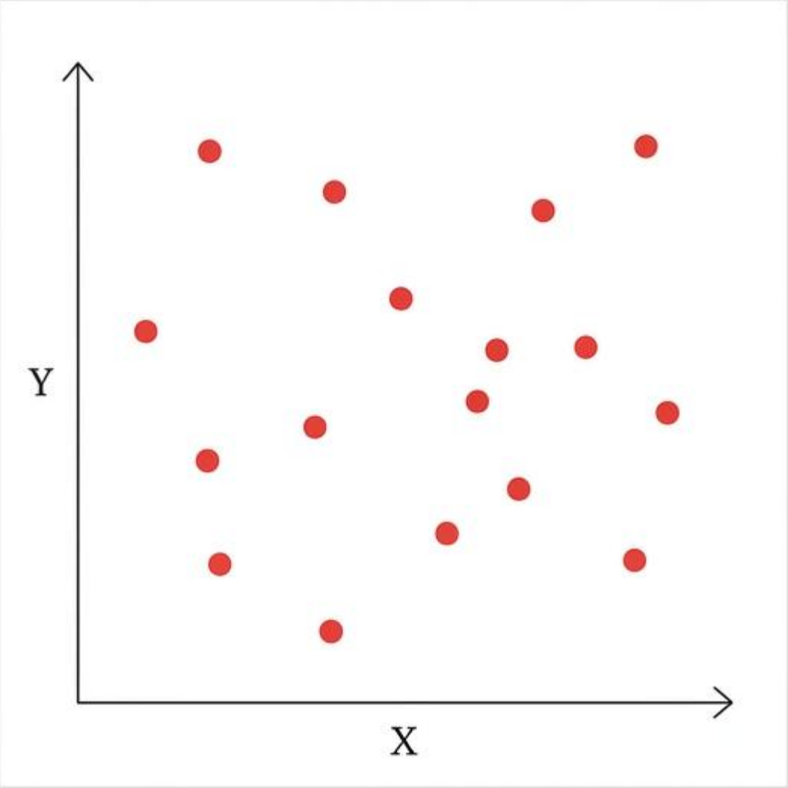
A Mecânica

Em vez de apenas clonar linhas (o que gera overfitting), o algoritmo cria novos dados sintéticos interpolando vizinhos da classe minoritária.

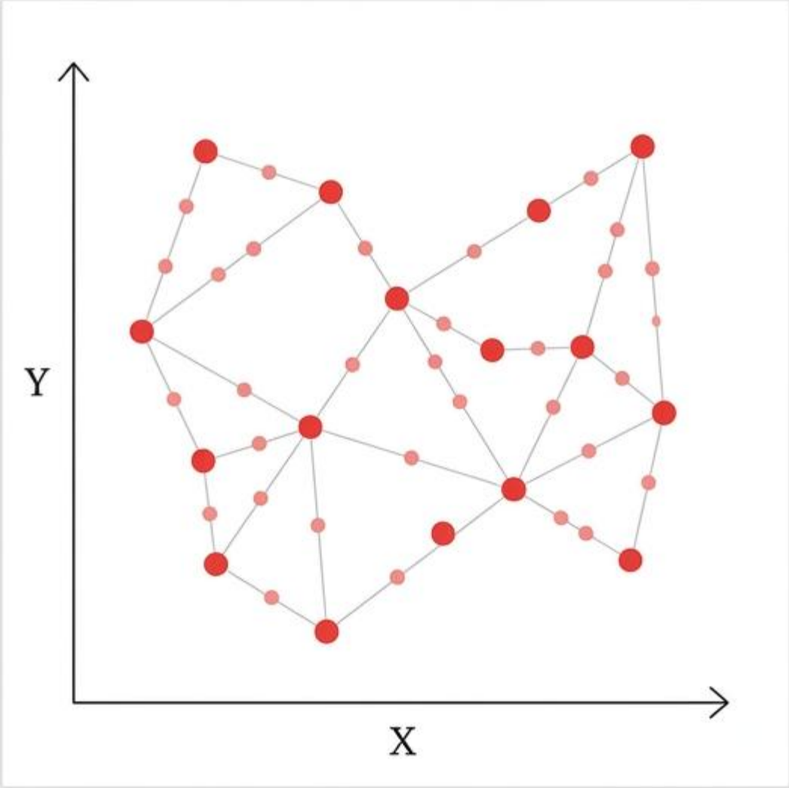
```python
from imblearn.over_sampling import SMOTE

# Instanciando o gerador sintético
smote = SMOTE(k_neighbors=4)

# Gerando novas fraudes artificiais para equilibrar
X_res, y_res = smote.fit_resample(X_treino, y_treino)
```



1. PONTOS ORIGINAIS ISOLADOS (2D)



2. PONTOS SINTÉTICOS AUMENTADOS



Hands-on: O Modelo Balanceado

- A Missão: lab_12_scaling_smote.ipynb
  - **1. Split Seguro:** Isolar X e y e dividir em Treino/Teste rigorosamente.
  - **2. Escala:** Aplicar o StandardScaler (lembrando a regra de ouro: apenas transform no teste).
  - **3. Equilíbrio:** Rodar o SMOTE no conjunto de treino e validar o 50/50 utilizando o método .value_counts().

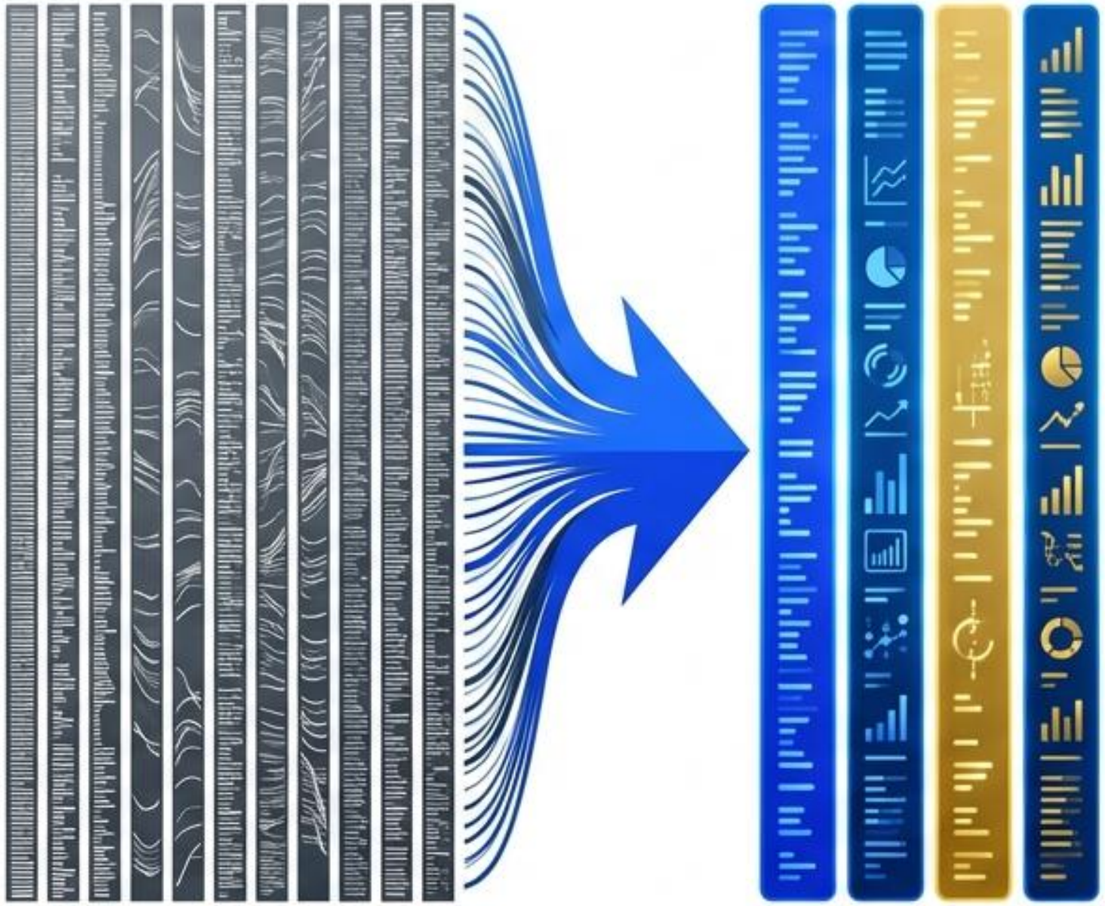
```
y_treino_smote.value_counts()
------------------------------
Fraude    5000
Normal    5000
Name: class, dtype: int64
```

Objetivo Concluído: O campo de jogo está nivelado.

A Maldição da Dimensionalidade

- O excesso de variáveis (features) dilui o poder preditivo e gera ruído estatístico.
- Custo exponencial de memória RAM e lentidão no processamento.
- Distorção no cálculo de distâncias geométricas pelos algoritmos.

- **Solução**: Redução de dimensionalidade focada na essência analítica.



As Três Famílias de Seleção de Features

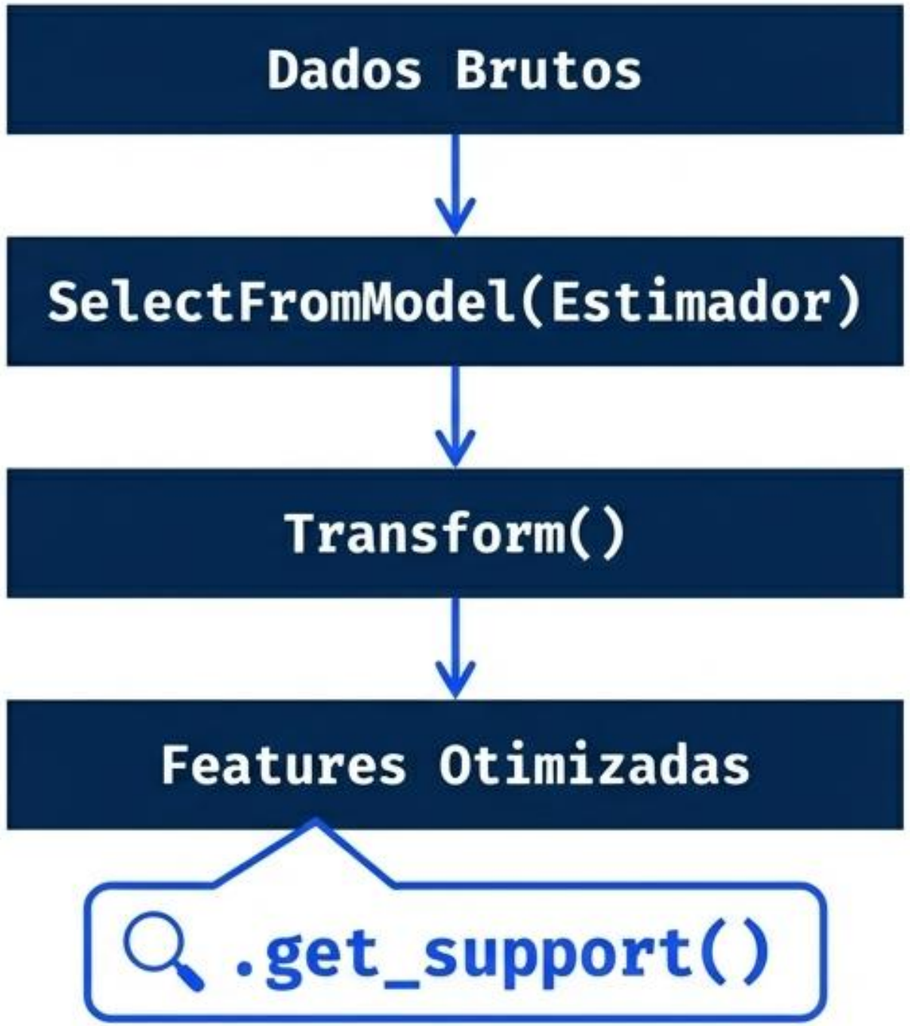
- **Métodos Intrínsecos:** Algoritmos que realizam a seleção internamente durante o treinamento (ex: Árvores de Decisão).

- **Métodos de Filtro:** Testes estatísticos prévios que avaliam a relação da variável com o alvo, independentes do modelo.

- **Métodos Wrappers:** Treinos iterativos e computacionalmente caros para encontrar o subconjunto matematicamente perfeito.

API Scikit-Learn: SelectFromModel

- Delega a decisão de importância a um modelo estimador base (como `Random Forest`).
- Seleciona features cujos pesos superam um limite matemático (`threshold`).
- Eficiência algorítmica: filtra os dados em uma única passagem de treinamento.
- Atributo `.get_support()` revela a máscara booleana das colunas sobreviventes.

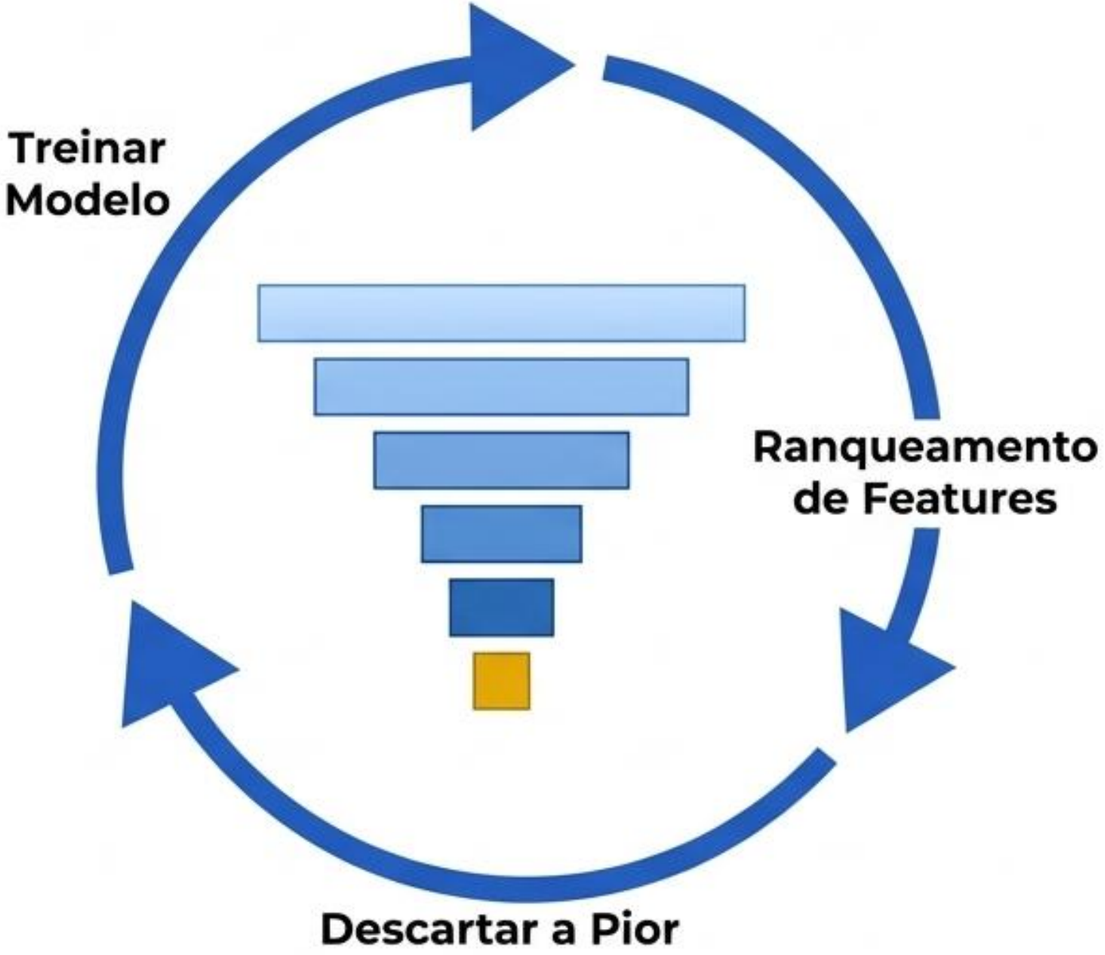


Usamos aqui o uma Árvore de decisão

```
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestClassifier

# 1. Usamos um modelo intrínseco como "juiz"
seletor =
SelectFromModel(RandomForestClassifier(n_estimators=100))

# 2. Cortamos as colunas irrelevantes
X_treino_limpo = seletor.fit_transform(X_treino, y_treino)
```

Eliminação Recursiva de Atributos (RFE)

- Representante clássico da família de seleção **Wrapper**.
- Elimina iterativamente o atributo mais fraco até atingir o número `n_features_to_select`.
- Exige alto poder computacional, mas garante precisão cirúrgica.
- Ideal para a peneira fina no final do pipeline de dados.

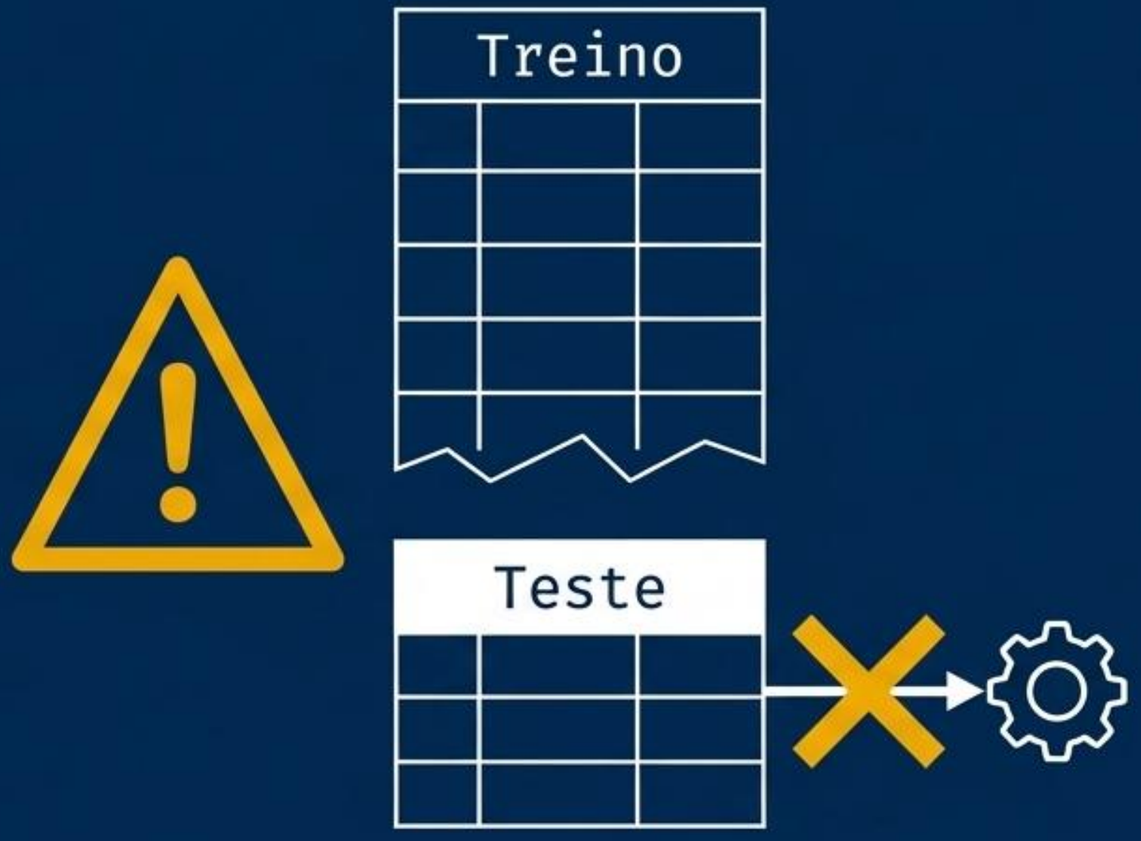


Eliminação Recursiva de Atributos (RFE)

```python
from sklearn.feature_selection import RFE

# RFE: Seleção de trás para frente (Backward)
rfe = RFE(estimator=modelo_juiz, n_features_to_select=15)
X_rfe = rfe.fit_transform(X_treino, y_treino)
```

Check de Aprendizado: A Ordem das Operações

- **Cenário:** Você tem 500 colunas criadas e decide usar o **RFE** para selecionar as 20 melhores.
- **A Pergunta:** Você deve rodar o algoritmo de Feature Selection antes ou depois do **`train_test_split()`?**
- O risco iminente de **Data Leakage (Vazamento de Dados).**



Hands-on: O Filtro Final (Labs 14 e 15)

1

- **1. Carga**: Importar o histórico de variáveis criadas.

2

- **2. O Juiz**: Aplicar `SelectFromModel` (Lab 14) e analisar o `.get_support()`.

3

- **3. A Peneira Fina**: Executar `RFE` (Lab 15) exigindo apenas as 10 features supremas.

4

- 4. **Validação**: Imprimir o `.shape` antes e depois para comprovar a redução.

O Fim do Data Prep e o Horizonte de Software

- **Entregável:** Labs 12 (Scaling), 14 e 15 (Selection) concluídos.
- **A Limitação:** Código solto e procedural em Jupyter Notebooks.
- **O Próximo Passo:** Transição de Cientista para Engenheiro de Dados (Aula 13).
- Empacotamento em **Pipelines Contínuos** orientados a objetos.

Dúvidas?

CRIATIVIDADE
INOVAÇÃO
¡DEIA!
COMAMUNIDADE
CRIATIVIDADE
BRAINSTORM
INOVAÇÃO

ibmec

A direção do amanhã.

/ibmec